Software Development Project — G&G City · Lab Backlog Sprint 1

LAB C1: The Cat Maker — Rawad Al-Juhaili

Topic: Building a Layered SVG Character + Animating It with CSS

Learning outcomes

By the end of this lab, students will be able to:

- Split an SVG drawing into named layers using `<g id="...">`
- Animate each layer with `@keyframes`, `transform-origin` and `transform-box`
- Wrap a component in a factory function `createCat()` that exposes a small API

🎯 Problem Scenario

G&G City needs a guide cat that walks between stations. Other members build on top of your cat:

- **C2 (Adnan) adds skills to it: `charge()`, `spell(ok)`, `say(text)`**
- **S3 (Ayman) places it inside every station in `index.html`**
- **The cat is inspired by Octocat in your own style, not a copy of GitHub's logo**

Your files: `cat/cat.svg, cat/cat.js, cat/test.html`

Required Outputs

Build:

1. **`cat.svg` with exactly 7 layers: `cat-tail, cat-leg-back, cat-leg-front, cat-body, cat-arm, cat-head, cat-eyes`**
2. **Idle + walk animations in CSS (idle: breathe, blink, tail sway · walk: legs swing opposite, body bobs)**
3. **`createCat(container, svgUrl)` → `{ el, ready, idle(), walk(), stop() }`**
4. **`test.html` with three buttons: idle, walk, stop**

Code is given only for the hard parts. Parts marked 📝 **Note** are easy: follow the note and write them yourself. Test with **Live Server** in VS Code (`fetch` does not work when you double-click a file).

Model Answer:

Task 1: Draw the cat in layers

> 📝 **Note (no code needed):** Draw the cat in any tool (Figma, Inkscape, Illustrator) or generate it with AI, then export as SVG. Put every moving part in its own group and give it the exact id from the list. Order matters: the first group is drawn at the back, so the tail comes first and the head last. Keep the eyes **inside** the head group so they move with it. Use a `viewBox="0 0 200 240"` and no fixed width/height.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 240">
  <g id="cat-tail">…</g>
  <g id="cat-leg-back">…</g>
  <g id="cat-leg-front">…</g>
  <g id="cat-body">…</g>
  <g id="cat-arm">…</g>
  <g id="cat-head">
    …
    <g id="cat-eyes">…</g>
  </g>
</svg>
```

Task 2: Animate the layers (hard part)

The trick is `transform-box: fill-box`: without it, every rotation turns around the corner of the whole SVG instead of the leg itself.

```
const CAT_CSS = `
  .gg-cat svg { width: 100%; height: auto; overflow: visible; }
  .gg-cat g   { transform-box: fill-box; }            /* origins relative to each layer */

  .gg-cat #cat-body     { transform-origin: 50% 100%; }
  .gg-cat #cat-head     { transform-origin: 50% 100%; }
  .gg-cat #cat-eyes     { transform-origin: 50% 50%; }
  .gg-cat #cat-tail     { transform-origin: 0% 100%; }
  .gg-cat #cat-leg-front,
  .gg-cat #cat-leg-back { transform-origin: 50% 0%; }

  /* Task 3: idle = breathe + blink + slow tail sway */
  .gg-cat.is-idle #cat-body { animation: cat-breathe 3s ease-in-out infinite; }
  .gg-cat.is-idle #cat-head { animation: cat-nod 3s ease-in-out infinite; }
  .gg-cat.is-idle #cat-tail { animation: cat-sway 3s ease-in-out infinite; }
  .gg-cat #cat-eyes         { animation: cat-blink 4s infinite; }

  /* Task 4: walk = legs swing in opposite directions + body bob */
  .gg-cat.is-walk #cat-leg-front { animation: cat-step .5s ease-in-out infinite alternate;
}
  .gg-cat.is-walk #cat-leg-back  { animation: cat-step .5s ease-in-out infinite alternate-
reverse; }
```

```
  .gg-cat.is-walk #cat-body,
  .gg-cat.is-walk #cat-head      { animation: cat-bob .25s ease-in-out infinite alternate;
}
  .gg-cat.is-walk #cat-tail      { animation: cat-sway .5s ease-in-out infinite; }

  @keyframes cat-breathe { 50% { transform: scale(1.03, .98); } }
  @keyframes cat-nod     { 50% { transform: translateY(2px); } }
  @keyframes cat-sway    { 50% { transform: rotate(10deg); } }
  @keyframes cat-blink   { 0%, 94%, 100% { transform: scaleY(1); } 97% { transform:
scaleY(.1); } }
  @keyframes cat-step    { from { transform: rotate(-16deg); } to { transform:
rotate(16deg); } }
  @keyframes cat-bob     { to { transform: translateY(-4px); } }
`;
```

Task 3: The createCat factory (hard part)

The SVG must be put **inline** in the page (not as <img>), otherwise CSS cannot reach its layers. `insertAdjacentHTML` is used instead of `innerHTML` so C2's speech bubble is not erased.

```
function createCat(container, svgUrl = 'cat/cat.svg') {
  injectStyle();

  const el = document.createElement('div');            // wrapper we control
  el.className = 'gg-cat';
  container.appendChild(el);

  // Load the SVG inline so CSS can reach its layers by id
  const ready = fetch(svgUrl)
    .then(res => {
      if (!res.ok) throw new Error('cat.svg not found at ' + svgUrl);
      return res.text();
    })
    .then(svg => { el.insertAdjacentHTML('afterbegin', svg); });   // keep C2's bubble

  function setMode(mode) {                             // 'idle' | 'walk'
    el.classList.remove('is-idle', 'is-walk');
    el.classList.add('is-' + mode);
    el.dataset.mode = mode;
  }

  const cat = {
    el,
    ready,                                             // await cat.ready if you need the
SVG
    idle() { setMode('idle'); return cat; },           // Task 3
    walk() { setMode('walk'); return cat; },           // Task 4
    stop() { return cat.idle(); }                      // back to standing
  };
```

```
  addSkills(cat);                                   // C2 plugs in here
  cat.idle();                                       // default state
  return cat;
}
```

Task 4: test.html

> 📝 **Note (no code needed):** A page with an empty `<div id="stage">`, three buttons, and `<script src="cat.js">`. Create the cat with `createCat(stage, 'cat.svg')` (path relative to the test page), and connect each button to `cat.idle()`, `cat.walk()`, `cat.stop()`. Open it with **Live Server**, not by double-clicking.

Expected Output



Page opens → idle



After walk() → legs swing

Submission

```
git pull
git add cat/
git commit -m "إضافة القط وحركتي الوقوف والمشي"
git push
```